

A PROJECT REPORT ON

**DYNAMIC RESOURCE ALLOCATION AND HIGH AVAILABILITY: DESIGNING
SCALABLE INFRASTRUCTURE VIA AMAZON AWS AUTO SCALING**

Submitted in partial fulfilment of the requirements for the degree of
Bachelor of Computer Applications (BCA)

Submitted By: Jay Singh Rajpurohit

Class: BCA 3rd Year

Roll No: [Insert Your University Roll Number]

Enrollment No: [Insert Your Enrollment Number]

**Under the Supervision of: [Insert Guide Name] Assistant Professor,
Department of Computer Applications**

**Host Organization for Internship: [Insert Training Company Name] Summer
Internship Program**

Department of Computer Applications

GOVERNMENT ENGINEERING COLLEGE BIKANER (ECB)

Year: 2026

CANDIDATE'S DECLARATION

I, Jay Singh Rajpurohit, student of Bachelor of Computer Applications (BCA) 3rd Year at Government Engineering College Bikaner (ECB), hereby declare that the project report entitled "Dynamic Resource Allocation and High Availability: Designing Scalable Infrastructure via Amazon AWS Auto Scaling" submitted by me is an authentic record of my own work.

This work was conducted during my Summer Internship Program and serves as a core requirement for the completion of my final year degree. The information, empirical analysis, and structural paradigms presented in this report have not been submitted previously to this or any other institution for the award of any academic degree, diploma, or fellowship.

All primary configuration steps, scripts, architectural designs, and deployment scenarios detailed within this manuscript were executed by me within the cloud environments provided during my industrial training program. Where assistance from external literature, documentation, or tutorials has been utilized, it has been duly acknowledged and cited within the references section.

Date: June 17, 2026

Place: Bikaner

Jay Singh Rajpurohit

Department of Computer Applications

Government Engineering College Bikaner

BONAFIDE CERTIFICATE

This is to certify that the project report entitled "**Dynamic Resource Allocation and High Availability: Designing Scalable Infrastructure via Amazon AWS Auto Scaling**" is a bonafide record of the technical work carried out by **Jay Singh Rajpurohit** under my direct guidance and internal supervision during the academic year 2025–2026.

The student has successfully completed the required training components of the Summer Internship Program and has demonstrated academic proficiency in Cloud Computing architectures, infrastructure-as-a-service frameworks, and AWS deployment strategies.

The evaluation committee has examined this project report during the final presentation sessions and finds it satisfactory for submission in partial fulfillment of the requirements for the degree of Bachelor of Computer Applications.

Internal Guide / Project Supervisor

Department of Computer Applications
Government Engineering College Bikaner

Head of Department (HOD)

Department of Computer Applications
Government Engineering College Bikaner

ACKNOWLEDGMENTS

First and foremost, I wish to express my deepest gratitude to **Government Engineering College Bikaner (ECB)** for providing an excellent academic environment and the technical infrastructure necessary to pursue my Bachelor of Computer Applications degree. The resources made available by the institution allowed me to explore advanced areas of computer science and systems engineering.

I am profoundly indebted to my project supervisor, whose continuous guidance, structured critiques, and insightful discussions significantly shaped the direction and execution of this cloud infrastructure report. Their patience, dedication, and technical expertise allowed me to deeply understand the operational challenges of modern cloud systems.

I also extend my sincere thanks to the training coordinators of the Summer Internship Program. The hands-on platform implementation sessions provided me with practical exposure to the Amazon Web Services console, command-line interfaces, and systems automation tools. Without their industry-aligned instruction, bridging the gap between theoretical computer science and cloud architecture would not have been possible.

Finally, I am incredibly grateful to my family and peers for their constant encouragement and support throughout my academic journey and technical training sessions. Their backing kept me motivated during long configuration and testing cycles.

ABSTRACT

Modern enterprise web applications operate in highly unpredictable environments where user traffic levels fluctuate rapidly. Traditional on-premises hardware solutions require significant capital expenditure to build out peak capacity, which frequently sits idle during off-peak hours. This report presents a modern solution built on **Amazon Web Services (AWS) Auto Scaling** to manage dynamic workloads efficiently.

The project examines how systems evolve from traditional hardware virtualization to public cloud Infrastructure-as-a-Service (IaaS) models. Using hands-on methods developed during a summer internship framework, an automated infrastructure loop was built using Amazon Elastic Compute Cloud (EC2), Application Load Balancers (ALB), and Amazon CloudWatch.

The system was subjected to simulated workload spikes to evaluate the performance of target-tracking and step-scaling policies. The practical results demonstrate that AWS Auto Scaling removes manual operational overhead, ensures high availability across multi-Availability Zone configurations, and minimizes costs by automatically decommissioning idle infrastructure instances in real time.

Ultimately, this study serves as a comprehensive reference blueprint for designing highly available, cost-effective, and fully automated application architectures on modern public cloud platforms.

TABLE OF CONTENTS

• Candidate's Declaration	ii
• Bonafide Certificate	iii
• Acknowledgments	iv
• Abstract	v

CHAPTER 1: INTRODUCTION TO CLOUD COMPUTING AND AWS

• 1.1 Evolution of Cloud Computing	1
• 1.2 Five Essential Characteristics of Cloud Environments	3
• 1.3 Cloud Service Models (IaaS, PaaS, SaaS)	5
• 1.4 Cloud Deployment Models	7
• 1.5 Introduction to Amazon Web Services (AWS)	9
• 1.6 AWS Global Infrastructure: Regions and Availability Zones	10
• 1.7 Core AWS Infrastructure Service Catalog	12

CHAPTER 2: LITERATURE REVIEW: VIRTUALIZATION VS. CLOUD

• 2.1 Theoretical Foundations of Hardware Virtualization	14
• 2.2 Hypervisor Classification and System Architecture	16
• 2.3 Hardware Virtualization vs. Containerization Frameworks	18

- 2.4 Structural Comparison: Virtualization and Cloud Computing 20
- 2.5 Strategic Benefits of Cloud Models over Traditional Datacenters 22

CHAPTER 3: DETAILED STUDY OF AMAZON AUTO SCALING

- 3.1 The Fundamental Physics of Elasticity in Cloud Architectures 23
- 3.2 Horizontal vs. Vertical Scaling Mechanics 24
- 3.3 Core Metrics Governing Infrastructure Scaling Triggers 25
- 3.4 Technical Anatomy of AWS Launch Templates 26
- 3.5 Auto Scaling Group (ASG) Parameter Mechanics 27
- 3.6 Lifecycle Hooks and Grace Period Configurations 28

CHAPTER 4: AUTO SCALING ARCHITECTURE, FEATURES, AND SECURITY

- 4.1 System Architecture and Feedback Loop Flow 29
- 4.2 Application Load Balancer (ALB) Routing Mechanics 30
- 4.3 CloudWatch Integration and Diagnostic Probing 31
- 4.4 Advanced Policies: Target Tracking and Step Scaling 32
- 4.5 Scheduled and Predictive Scaling utilizing Machine Learning 33
- 4.6 IAM Frameworks and Network Isolation Security 34
- 4.7 Enterprise Production Scenarios and Industry Use Cases 35

CHAPTER 5: PRACTICAL IMPLEMENTATION AND CONFIGURATION

- 5.1 Project Environment Requirements and Network Layout
..... 36
- 5.2 Step-by-Step Production Security Group Creation
..... 37
- 5.3 Provisioning the Application Load Balancer Tier
..... 38
- 5.4 Authoring the Launch Template with Scripted User Data
..... 39
- 5.5 Deploying and Binding the Auto Scaling Group
..... 40
- 5.6 Workload Simulation and Stress-Testing Execution
..... 41
- 5.7 Analyzing Live Scale-Out Logs and CloudWatch Metrics
..... 42

CHAPTER 6: ADVANTAGES, LIMITATIONS, AND FUTURE SCOPE

- 6.1 Cost Optimization and Operational Advantages
..... 43
- 6.2 Architectural Vulnerabilities, Cooldown Caps, and Thrashing
..... 44
- 6.3 Stateless vs. Stateful Application Disruption Barriers
..... 45
- 6.4 Next-Generation Systems: Proactive AI-Driven Cloud Scaling
..... 46

CHAPTER 7: CONCLUSION AND REFERENCES

- 7.1 Comprehensive Technical Project Synthesis
..... 47
- 7.2 Future Scope for Systems Research
..... 48
- 7.3 Formal Academic Bibliography and References
..... 49

LIST OF TABLES & FIGURES

- **Table 1.1:** Comparative Service Matrix Matrix
..... 5
- **Table 2.1:** Hypervisor Structural Characteristics
..... 16
- **Table 2.2:** Virtualization vs Cloud Technical Breakdown
..... 20
- **Figure 2.1:** Bare-Metal vs. Hosted Architecture
..... 17
- **Figure 4.1:** Auto Scaling Infrastructure Loop Architecture
..... 29
- **Figure 5.1:** CloudWatch Metric Trigger Behavior Diagram
..... 42

CHAPTER 1: INTRODUCTION TO CLOUD COMPUTING AND AWS

1.1 Evolution of Cloud Computing

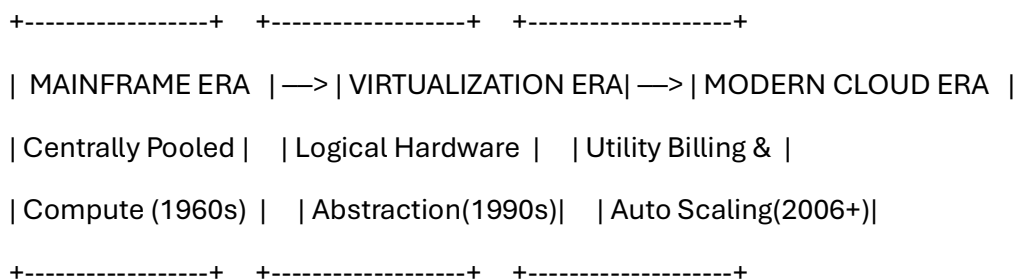
The shift to cloud computing represents a fundamental change in how corporate enterprise data architectures are structured worldwide. Historically, organizations had to build and maintain their own on-premises computing infrastructure. This model required substantial capital investment to buy physical server racks, build complex raised-floor environments, and install heavy-duty uninterruptible power supplies (UPS) alongside specialized industrial HVAC cooling units.

Furthermore, network administrators had to manually handle physical switches, storage area networks (SAN), and security firewalls. This hardware-bound model limited business agility, as provisioning new computing power often took weeks or months due to long procurement, configuration, and testing pipelines.

In the early 2000s, virtualization technologies and fast, widespread internet access changed this landscape, leading to the modern utility-based cloud computing model. Cloud computing treats computing power like electricity or water. Instead of building a private power grid, organizations plug directly into a shared network and pay only for the exact amount of resources they consume.

By moving away from fixed capital expenditures (CapEx) and toward flexible operating expenditures (OpEx), companies can launch new software applications almost instantly without hardware constraints. This democratizes enterprise-grade infrastructure, enabling small startups to use the same highly available global networks as multinational corporations.

The historical journey toward cloud computing can be broken down into several distinct eras, each building on the technological breakthroughs of the previous one:



During the Mainframe Era of the 1960s, large computing systems used time-sharing methodologies to let multiple users access central processors. However, these systems lacked flexibility and required strict physical proximity to the

terminals. The client-server models of the 1980s distributed computing across corporate networks, but led to isolated hardware infrastructure where individual servers were tied to single applications, creating massive inefficiencies.

In the late 1990s, the widespread adoption of bare-metal hypervisors completely changed data center management. Virtualization allowed multiple logical operating systems to run safely on a single physical host machine. This paved the way for cloud hyperscalers to build large, multi-tenant data facilities that offer computing resources over the public internet, completely shifting infrastructure management from a hardware discipline into a software-driven process.

1.2 Five Essential Characteristics of Cloud Environments

According to the National Institute of Standards and Technology (NIST), true cloud computing architectures are defined by five core characteristics that separate them from standard legacy networks:

1. **On-Demand Self-Service:** Users can provision computing assets, such as server runtimes, storage space, and network gateways, automatically via web dashboards or programmatic APIs. This process happens instantly without requiring human intervention from the cloud service provider's engineering staff.
2. **Broad Network Access:** Cloud platform capabilities are available over public or private networks and can be accessed seamlessly from standard client devices, including desktop workstations, laptops, tablets, and smartphones, using standard protocols.
3. **Resource Pooling:** The provider's physical computing resources are pooled together to serve multiple customers using a multi-tenant architecture. Physical compute cores, memory blocks, and storage allocation dynamically shift based on real-time customer demand, keeping the underlying hardware layer invisible to the end user.
4. **Rapid Elasticity:** Cloud capabilities can be scaled up or down almost instantly. To users, the available computing power appears virtually limitless, allowing them to scale resources up to match unexpected traffic surges and scale back down immediately when demand drops.
5. **Measured Service:** Cloud resource usage is tracked, metered, and optimized automatically using monitoring systems. This structural transparency ensures customers are billed accurately based on the exact amount of storage, processing, or bandwidth they consume.

To understand why these five characteristics are so impactful, we must look at how they change daily systems engineering operations:

Under the traditional hardware infrastructure model, adding a server required raising an internal purchase order, getting budget approvals, waiting for shipping, unboxing the server, mounting it in a physical rack, running Ethernet cables, and manually installing the operating system. This process was slow and error-prone.

TRADITIONAL PROVISIONING CYCLE (Weeks to Months):

[Approve Budget] -> [Order Server] -> [Ship Hardware] -> [Physical Rack Mount] -> [OS Install]

CLOUD PROVISIONING CYCLE (Seconds):

[API Call / Web Console Click] -> [Instance Ready & Serving Traffic]

Cloud computing replaces this entire physical cycle with instant, software-driven provisioning. Because resources are managed as code, an engineer can use an API call or web console click to launch hundreds of virtual servers simultaneously across different global data centers.

The underlying resource pooling architecture makes this possible by maintaining large farms of active physical hardware. A smart orchestration layer handles isolation and customer provisioning automatically, turning hardware management into a seamless utility service.

1.3 Cloud Service Models (IaaS, PaaS, SaaS)

Cloud computing services are organized into three primary delivery models based on the level of control, flexibility, and management they offer:

Service Model	Definition	Key Responsibilities Managed by Provider	User Responsibility
Infrastructure as a Service (IaaS)	Provisioning of fundamental computing resources, networking, firewalls, and storage.	Physical hardware, virtualization layer, facility security, global networking.	Operating system, middleware, runtime environments, applications, data configurations.
Platform as a Service (PaaS)	Provisioning of execution environments where users can deploy apps without managing underlying OS.	Hardware, OS patching, runtime containers, database scaling engines.	Application code writing, logic optimization, data schemas.
Software as a Service (SaaS)	Delivery of fully functional, end-user applications hosted and managed entirely by a vendor.	End-to-end management from physical layer up to application code.	Basic client-side settings, user access control.

Choosing the right service model is a critical decision that impacts an organization's development speed, operational overhead, and overall engineering flexibility:

+-----+

| SAAS | Managed: Hardware, OS, Runtime, Data, Application | <- Lowest Overhead

+-----+

| PAAS | Managed: Hardware, OS, Runtime | User: App & Data |

+-----+

| IAAS | Managed: Hardware, Hypervisor | User: OS, App, Data| <- Highest Control

+-----+

- **IaaS (e.g., Amazon EC2):** Gives systems administrators maximum control over their infrastructure environment. It functions exactly like a raw physical server, allowing teams to install custom operating system kernels, configure complex local firewalls, and deploy specialized monitoring tools. This flexibility is ideal for legacy application migrations but requires ongoing maintenance like OS patching and security updates.
- **PaaS (e.g., AWS Elastic Beanstalk):** Designed to streamline development workflows by removing infrastructure management tasks. Developers can upload their application code (such as Node.js, Python, or Java), and the PaaS platform handles provisioning, load balancing, and auto-scaling automatically.
- **SaaS (e.g., Microsoft 365):** Delivers a complete, ready-to-use software solution. Users focus entirely on interacting with the application, while the provider handles all underlying infrastructure maintenance, software updates, and data security.

1.4 Cloud Deployment Models

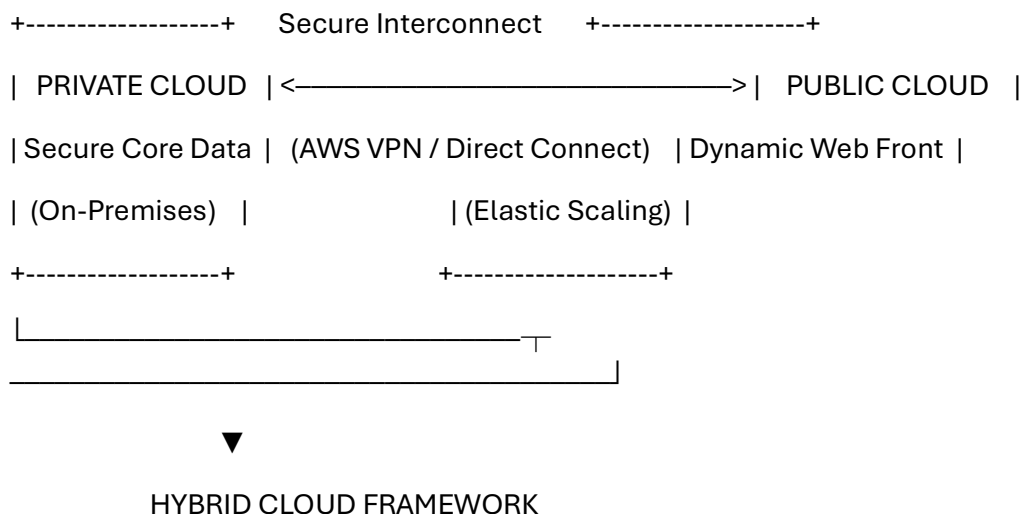
Cloud infrastructure setups vary depending on an organization's specific security, compliance, and operational requirements:

- **Public Cloud:** The cloud infrastructure is owned and operated by a third-party vendor (such as Amazon Web Services). It delivers computing resources over the public internet to multiple organizations using a

shared, multi-tenant architecture. This model provides maximum scalability, eliminates hardware maintenance costs, and uses a flexible, pay-as-you-go pricing model.

- **Private Cloud:** The cloud infrastructure is dedicated entirely to a single organization. It can be physically located at an on-site data center or hosted by a third-party provider. This model offers high levels of data privacy, predictable performance, and deep infrastructure control, making it ideal for industries with strict regulatory constraints like healthcare or finance.
- **Hybrid Cloud:** This approach connects public and private cloud environments using secure networking solutions (such as AWS Site-to-Site VPN or Direct Connect). It allows data and applications to move fluidly between environments, helping companies keep highly sensitive assets secure in a private cloud while running high-volume web services in the public cloud.

Selecting a deployment model requires balancing financial costs against required compliance, data security, and control levels:



The **Hybrid Cloud framework** is the preferred model for large, established enterprises. For example, a bank might choose to host its core financial ledger databases inside a private on-premises infrastructure to satisfy national data compliance regulations.

At the same time, it can deploy its consumer web application portals onto the public AWS cloud tier. This approach keeps sensitive data protected within the private network, while allowing the public web front-end to scale up or down

automatically using cloud auto-scaling tools during major shopping events or high-traffic periods.

1.5 Introduction to Amazon Web Services (AWS)

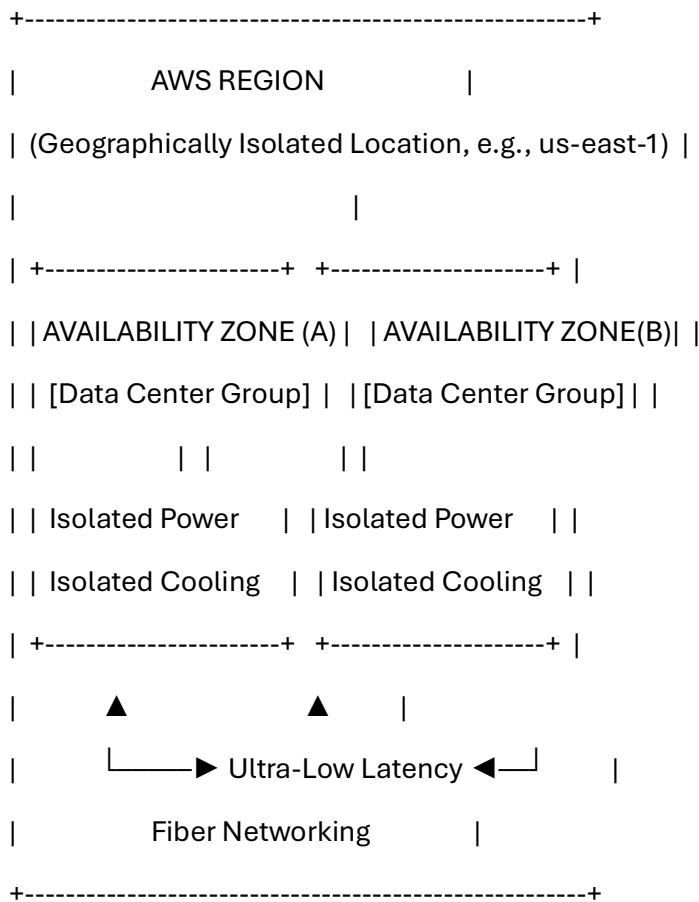
Launched commercially in 2006, Amazon Web Services (AWS) pioneered the public cloud market and has grown into a leading global cloud platform. AWS provides an ecosystem of over 200 fully featured services, ranging from basic compute and storage solutions to advanced database design, data analytics, machine learning, and Internet of Things (IoT) frameworks.

AWS succeeded by transforming complex hardware engineering into software-defined services accessible via simple HTTP API calls. Developers no longer need to wire physical servers or configure network switches. Instead, they can spin up enterprise-grade infrastructure environments in seconds using the AWS Management Console, Command Line Interface (CLI), or Infrastructure-as-Code (IaC) tools like AWS CloudFormation.

By removing the need for upfront capital costs, AWS allows organizations to build resilient, global applications efficiently, shifting the focus from hardware management to software innovation.

1.6 AWS Global Infrastructure: Regions and Availability Zones

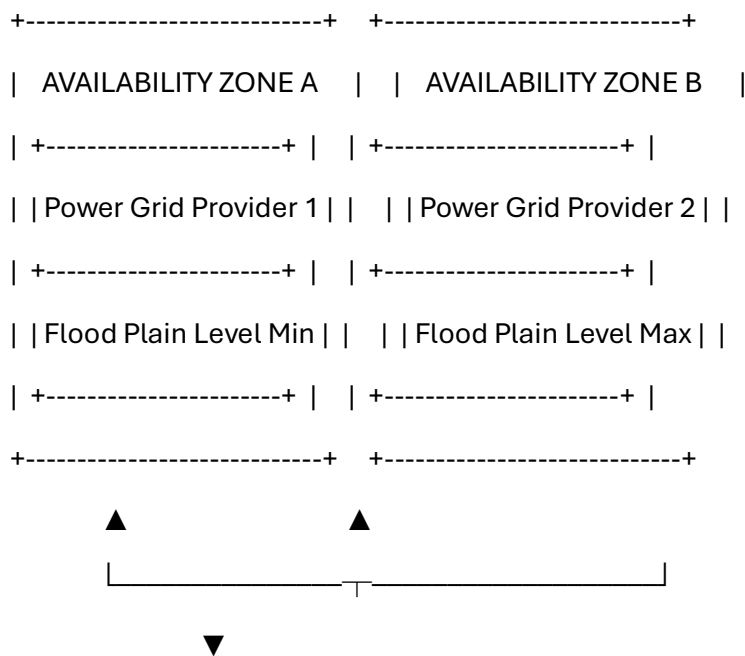
The reliability and high availability of AWS depend on its global physical infrastructure, which is built using a clear geographic hierarchy:



- **AWS Regions:** A Region is a distinct, geographically isolated area of the world (such as us-east-1 in Virginia or ap-south-1 in Mumbai) that houses clusters of data centers. Each Region operates independently to ensure that natural disasters or infrastructure failures in one area do not affect services in another.
- **Availability Zones (AZs):** Every AWS Region contains multiple, distinct Availability Zones. An AZ consists of one or more physical data centers equipped with independent, redundant power sources, industrial cooling systems, and physical security protection. AZs within a Region are connected to each other through ultra-low latency, private fiber-optic networks.

Designing applications to run across multiple AZs forms the foundation of modern, fault-tolerant cloud architecture.

To build truly resilient systems, engineers must understand how AWS isolates infrastructure faults using its global design patterns:



- **Storage - Amazon Simple Storage Service (S3):** An object storage platform designed to store and retrieve any amount of unstructured data from anywhere on the web. It features a highly redundant architecture that delivers 99.999999999% durability.
- **Networking - Amazon Virtual Private Cloud (VPC):** Allows users to provision a logically isolated section of the AWS cloud network. Administrators maintain full control over IP address ranges, subnets, routing tables, and network access control gateways.
- **Database - Amazon Relational Database Service (RDS):** Simplifies setting up, operating, and scaling relational database engines (such as MySQL, PostgreSQL, MariaDB, and Oracle) by automating administrative tasks like patching, backups, and failovers.

CHAPTER 2: LITERATURE REVIEW: VIRTUALIZATION VS. CLOUD COMPUTING

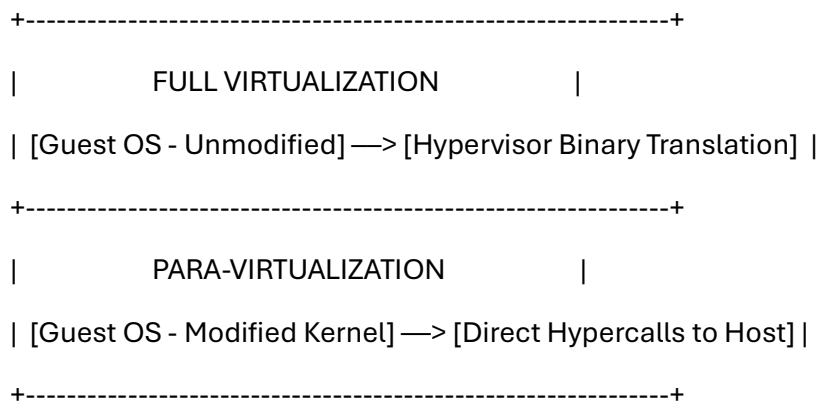
2.1 Theoretical Foundations of Hardware Virtualization

Before cloud hyper-scalers arrived, hardware virtualization was the primary technology used to maximize performance in physical corporate data centers. Historically, enterprise software applications ran directly on bare-metal servers. Because single applications rarely used a server's full processing capacity, this configuration led to low hardware utilization rates, often hovering between 5% and 15%. This meant valuable CPU power and memory sat idle, while organizations continued to pay high costs for power, cooling, and data center real estate.

Hardware virtualization resolves this inefficiency by inserting a thin software abstraction layer directly between the physical hardware and the operating systems. This abstraction layer tricks multiple guest operating systems into believing they each have exclusive access to their own physical CPU cores, memory blocks, storage arrays, and network cards.

By running multiple, isolated virtual environments on a single physical host machine, organizations can consolidate their hardware footprints, maximize utilization rates, lower power consumption, and simplify data center management.

Hardware virtualization uses several primary methods to handle abstractions based on performance requirements and system architectures:



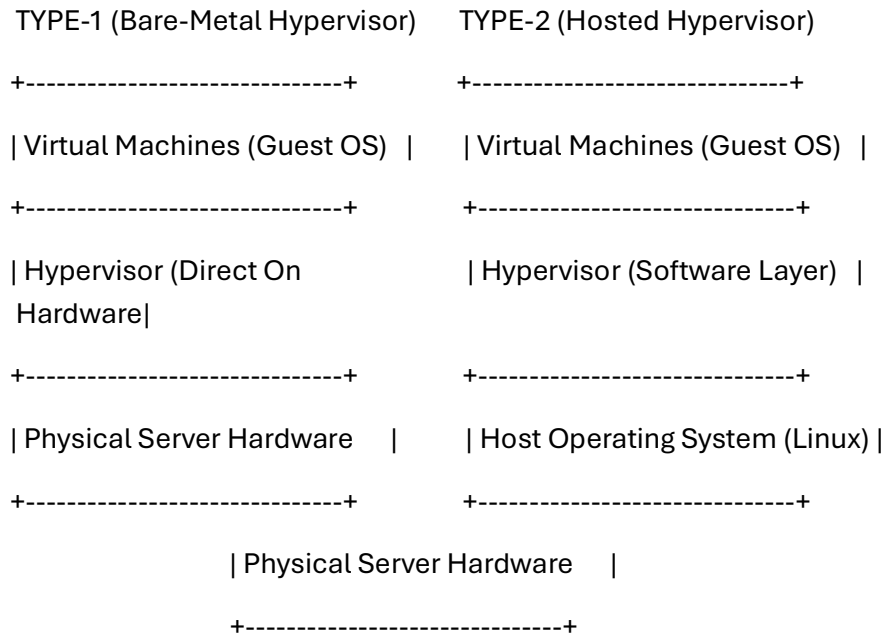
- Full Virtualization:** The guest operating system is completely unaware that it is running in a virtualized environment. The hypervisor intercepts every hardware request from the guest OS and translates it on the fly using binary translation techniques. This approach supports unmodified operating systems but introduces processing overhead.

- **Para-Virtualization:** The guest operating system's kernel is explicitly modified to be aware of the virtualization layer. Instead of executing raw hardware commands, the guest OS sends direct requests (called *hypercalls*) straight to the hypervisor. This eliminates binary translation overhead and improves performance, but requires access to the operating system's source code for modifications.

2.2 Hypervisor Classification and System Architecture

The core software engine responsible for managing virtualization abstractions is called the Hypervisor, or Virtual Machine Monitor (VMM). Hypervisors are classified into two architectural types based on where they sit in the systems stack:

Technical Parameter	Type-1 Hypervisor	Type-2 Hypervisor
Installation Target	Directly on bare-metal physical hardware.	As an application on top of an existing host OS.
Performance Profile	High performance with near-zero overhead.	Moderate performance due to host OS translation.
Primary Use Cases	Enterprise production datacenters and cloud platforms.	Local testing, software development, and labs.
Resource Isolation	Highly secure; hardware-enforced boundaries.	Dependent on the security features of the host OS.



Modern public cloud vendors like AWS have evolved hypervisor designs to improve performance and security. Traditionally, Type-1 hypervisors had to handle networking, storage management, and security controls directly within the host CPU software layer, which consumed computing power that could otherwise be allocated to customer virtual machines.

TRADITIONAL HYPERVISOR ARCHITECTURE:

[Host CPU] —> Handles Compute + Storage + Network + Security (High Overhead)

AWS NITRO ARCHITECTURE:

[Host CPU] —> Dedicated Exclusively to Customer VMs (Near 100% Performance)

|

▼ Offloaded via PCIe Interfaces

[Nitro ASIC Cards] —> Dedicated Hardware Handles Storage + Network + Security

To resolve this bottleneck, AWS developed the **Nitro System Architecture**. Nitro offloads networking, storage, management, and security tasks onto dedicated hardware micro-cards (ASICs). This removes the management burden from the

main host processor, allowing nearly 100% of the host CPU and memory to be dedicated to running customer EC2 virtual machines. This hardware-enforced isolation delivers bare-metal performance while maintaining the security benefits of virtualization.

2.3 Hardware Virtualization vs. Containerization Frameworks

As modern software engineering has evolved, containerization has emerged alongside hardware virtualization as a popular method for resource isolation. It is important to understand the structural differences between these two technologies:

- **Hardware Virtualization (Virtual Machines):** Virtualization abstracts an entire physical server, including the underlying hardware components. Each Virtual Machine (VM) includes its own copy of a guest operating system, virtual device drivers, application runtimes, and system files. While this design provides strong security boundaries, it requires significant system memory and results in slower boot times, as the entire guest OS must initialize from scratch.
- **Containerization (e.g., Docker Engine):** Instead of abstracting the hardware layer, containerization abstracts the operating system kernel itself. Multiple isolated containers run directly on a single host operating system and share its core kernel. Because containers do not carry a heavy guest OS payload, they are lightweight, scale out in milliseconds, use far less memory, and are ideal for microservices architectures.

The architectural differences between virtual machines and containers come down to how they enforce system boundaries within the processor's security rings:

VIRTUAL MACHINE ISOLATION (Hardware Enforced):

[Ring 0 - Hypervisor] —> [Ring 1 - Guest OS Kernel] —> [Ring 3 - User App]

* Strong, independent hardware abstraction boundaries.

CONTAINER ISOLATION (OS Kernel Enforced):

[Ring 0 - Host OS Kernel] —> [Namespaces / Cgroups Isolation] —> [Ring 3 - Container App]

* Shared kernel space; lightweight but shares software risk boundaries.

Virtual Machines rely on hardware-enforced isolation. The guest operating system kernel runs in an isolated processor space (Ring 1), separate from the hypervisor layer (Ring 0). This design makes it nearly impossible for a security breach inside one virtual machine to affect neighboring systems on the same physical host.

Containers, by contrast, use Linux kernel features like namespaces and cgroups to isolate processes within the same user space (Ring 3). While this makes containers fast and efficient, they share the underlying host operating system kernel, making them less isolated than virtual machines.

2.4 Structural Comparison: Virtualization and Cloud Computing

A common point of confusion in computer science is the relationship between virtualization and cloud computing. Virtualization is a software tool that manipulates physical hardware components, while cloud computing is an all-encompassing service model built on top of virtualization to deliver resources dynamically.

Technical Parameter	Traditional Virtualization	Cloud Computing (AWS)
Resource Provisioning	Manual intervention required by infrastructure engineers.	Fully automated self-service via APIs and consoles.
Elasticity Capability	Limited by the physical capacity of the on-site hypervisor node.	Highly elastic; near-infinite resource provisioning.
Financial Structuring	High upfront Capital Expenditure (CapEx) for hardware.	Variable Operational Expenditure (OpEx) billing.
Multi-Tenancy	Designed for single-organization workload isolation.	Native multi-tenant architecture with logical isolation.
Scalability Speed	Hours to weeks (if new hardware procurement is required).	Instantaneous execution via automated policies.

To understand this technical comparison in a business context, think of virtualization as an engine, and cloud computing as a fully managed vehicle service:

Virtualization allows data center managers to partition a large physical server into smaller virtual servers. However, it does not include self-service provisioning, utility-based automated billing, or instant elasticity. Engineers must still manually allocate resources, monitor performance, and manage infrastructure capacity.

VIRTUALIZATION ENGINE (Tooling):

[Hypervisor Allocation] + [Manual Capacity Configuration] = Local Resource Efficiency

CLOUD INFRASTRUCTURE (Ecosystem):

[Virtualization Engine] + [Self-Service APIs] + [Auto Scaling] + [Pay-as-you-go] = Cloud Utility

Cloud computing packages virtualization into a consumer-focused utility model. By adding automation layers, API controllers, and real-time monitoring tools on top of virtualized hardware, platforms like AWS turn infrastructure management into a fully automated service that responds to user demands dynamically.

2.5 Strategic Benefits of Cloud Models over Traditional Datacenters

Migrating from on-premises virtualization infrastructure to a public cloud platform like AWS provides clear strategic advantages:

1. **Elimination of Capacity Guessing:** In traditional data centers, systems administrators must purchase enough hardware to handle historical peak workloads, leaving expensive resources underutilized during low-traffic periods. AWS resolves this issue by letting companies provision infrastructure that scales up or down automatically in real time.
2. **Global Reach in Minutes:** Deploying application servers across different global markets using traditional hardware requires complex logistics and data center configurations. With AWS, administrators can replicate their infrastructure across multiple global Regions with a few clicks.
3. **Built-in Redundancy and Resilience:** Setting up multi-site data replication in a private data center is costly and technically complex. AWS builds high availability directly into its platform, making it easy to

distribute application traffic across multiple, physically isolated Availability Zones.

the physical hardware caps of the underlying host machine, and resizing an instance usually requires taking the server offline, causing application downtime.

- **Horizontal Scaling (Scaling Out / In):** This approach adds or removes individual server instances within a shared computing pool. Instead of buying a larger server, horizontal scaling connects multiple standard instances together behind a load balancer. This model offers near-infinite scalability and eliminates single points of failure, ensuring that if one server breaks, the remaining instances continue handling user requests without disruption. **Amazon EC2 Auto Scaling** uses this horizontal scaling design pattern.

To illustrate the operational differences between these two scaling models, consider the following technical breakdown:

VERTICAL SCALING (Scale Up):

[Small Server: 2 vCPU] —> [DOWNTIME WINDOW] —> [Large Server: 16 vCPU]

* Limited by host hardware caps; requires system restarts.

HORIZONTAL SCALING (Scale Out):

[Server 1] + [Server 2] —> [Traffic Spike] —> [Server 1] + [Server 2] + [Server 3]

* Scaled seamlessly without downtime; high availability by design.

Vertical scaling hits a physical limit when an instance reaches the maximum capacity of its host machine's motherboard sockets. Additionally, changing instance sizes requires modifying the underlying hardware profile, which causes application downtime during the reboot cycle.

Horizontal scaling bypasses these physical limits by spreading workloads across independent server nodes. Since instances are added or removed while the application is running, the platform can handle growing traffic volumes seamlessly without interrupting active user sessions.

3.3 Core Metrics Governing Infrastructure Scaling Triggers

To automate horizontal scaling, cloud platforms rely on continuous performance data gathered by monitoring services. Key metrics used to trigger scaling events include:

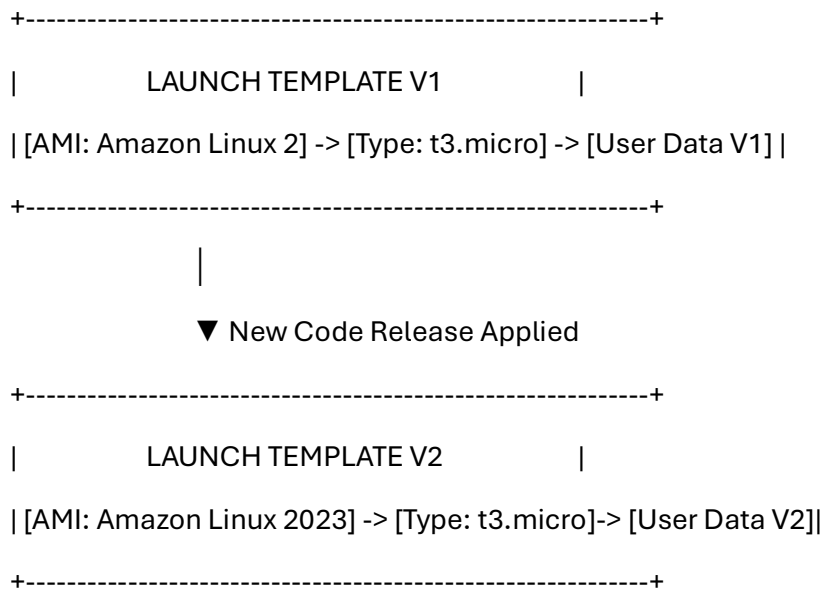
- **Average CPU Utilization:** Tracks the percentage of processing power being consumed across all active instances. If the average CPU usage stays above a set threshold (like 75%) for too long, it indicates the system is under heavy load and needs additional instances.
- **Memory Utilization:** Monitors RAM consumption across the instance pool, which is critical for memory-heavy applications like caching engines.
- **Network In / Network Out:** Measures the volume of data flowing through the network interfaces of the application servers. This metric helps identify bandwidth bottlenecks before they degrade performance.
- **Application Load Balancer Request Count per Target:** Tracks the average number of incoming HTTP/HTTPS requests hitting each active instance, allowing the system to scale out proactively based on direct traffic volumes.

3.4 Technical Anatomy of AWS Launch Templates

An **AWS Launch Template** serves as the configuration blueprint used by the Auto Scaling engine to spin up new EC2 instances automatically. It groups all the configuration settings required to ensure that every new server matches the existing application environment:

- **Amazon Machine Image (AMI) ID:** Specifies the base operating system, pre-installed software frameworks, and application code deployments.
- **Instance Type Configuration:** Defines the underlying compute, memory, network, and storage profile (such as selecting a general-purpose t3.micro instance).
- **Network Security Signatures:** Assigns specific Virtual Private Cloud (VPC) subnets and EC2 Security Groups to act as firewalls, controlling inbound and outbound traffic access.
- **Storage Device Mapping:** Outlines the configuration, size, and encryption rules for attached storage volumes (Amazon Elastic Block Store).
- **User Data Scripts:** Injects initialization scripts that run automatically when the instance boots, handling final software installations and application deployment tasks.

Unlike legacy launch configurations, modern Launch Templates provide advanced management features like configuration version control:

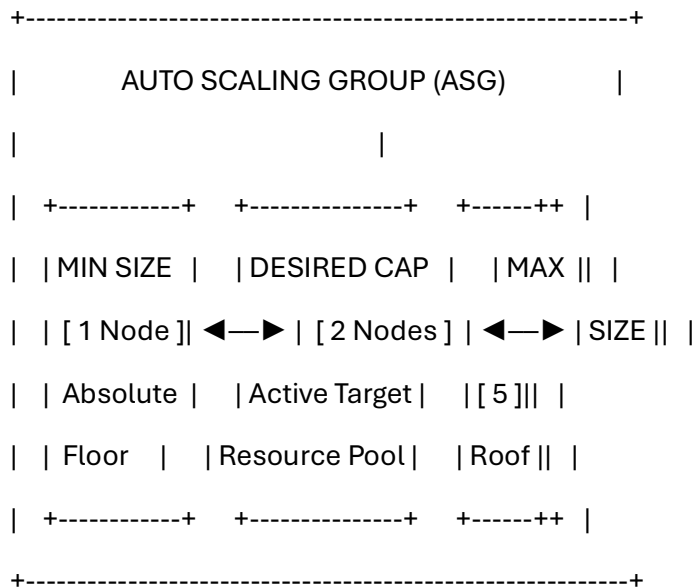


Launch Template versioning allows engineering teams to update infrastructure configurations safely. For example, when deploying a new version of an application, developers can save the updated code settings as Version 2 of the template.

If the new deployment encounters errors, administrators can instantly update the Auto Scaling Group to use Version 1 again. This capability simplifies deployment rollbacks and helps minimize application downtime during updates.

3.5 Auto Scaling Group (ASG) Parameter Mechanics

The **Auto Scaling Group (ASG)** is the primary management tool used to organize collections of EC2 instances and define their scaling boundaries. It relies on three capacity settings to manage instance counts:



- **Minimum Size:** The lowest number of active instances required to keep the application stable. The ASG will never let the instance count drop below this floor, even during zero-traffic windows.
- **Maximum Size:** The upper limit of compute instances allowed within the group. This setting protects organizations from runaway cloud costs caused by traffic spikes or denial-of-service attacks.
- **Desired Capacity:** The target number of operational instances the group should maintain. The scaling engine automatically changes this value in response to scaling triggers, launching or terminating instances to match the new target.

3.6 Lifecycle Hooks and Grace Period Configurations

Managing an automated server environment requires smooth instance transitions during scale-out or scale-in events:

- **Health Check Grace Period:** When a new instance launches, it needs time to boot up, install software, and download application files. The Health Check Grace Period tells the ASG to wait a specified time (such as

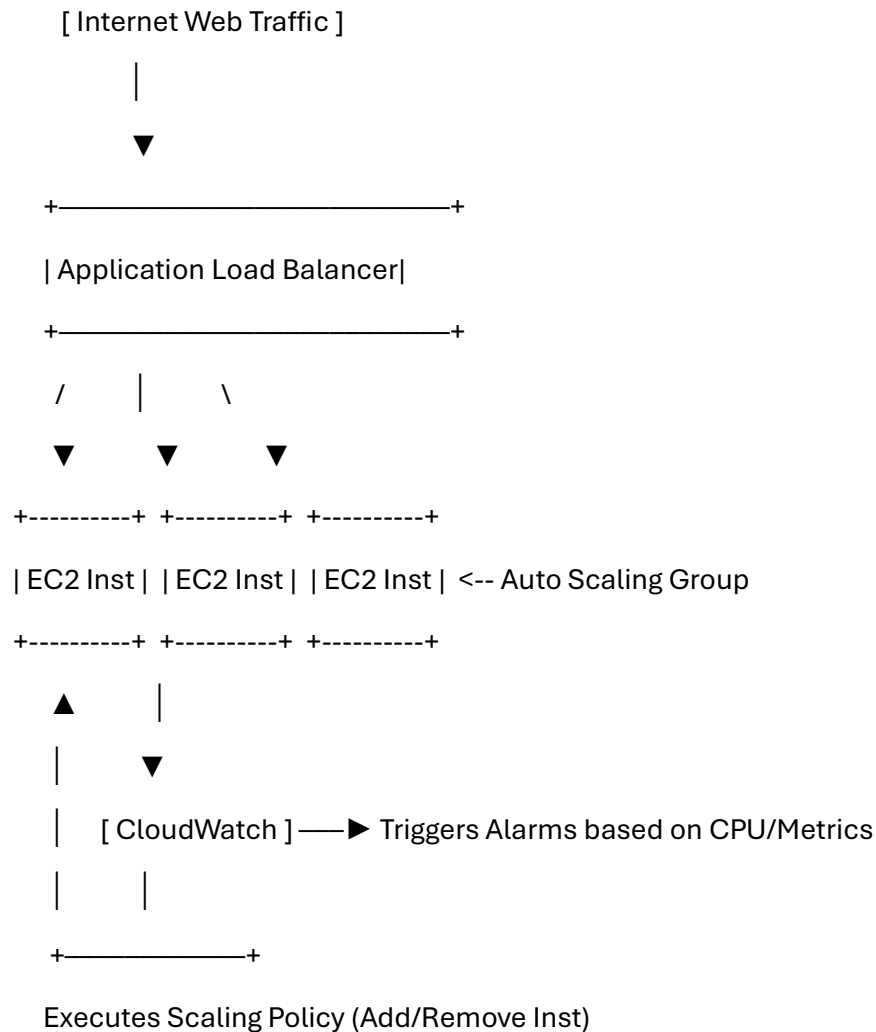
300 seconds) before checking the instance's health status, preventing the system from prematurely terminating a server that is simply still booting.

- **Lifecycle Hooks:** These hooks pause the standard launch or termination process, allowing administrators to run custom scripts before an instance enters or leaves service. For example, a termination hook can pause an instance's deletion for 10 minutes, giving the system time to extract log files or back up local data caches to an external storage bucket before the server is destroyed.

CHAPTER 4: AUTO SCALING ARCHITECTURE, FEATURES, AND SECURITY

4.1 System Architecture and Feedback Loop Flow

In production environments, Amazon EC2 Auto Scaling works alongside other core services to create an automated, closed-loop feedback system. The architecture relies on three primary components working together:



1. **The Traffic Routing Tier:** Managed by an Application Load Balancer, which acts as the single public entry point for all incoming user web requests.
2. **The Elastic Compute Tier:** Managed by the Auto Scaling Group, which hosts the application servers inside isolated private subnets across multiple Availability Zones.
3. **The Monitoring Tier:** Managed by Amazon CloudWatch, which continuously tracks performance metrics across all running instances.

4.2 Application Load Balancer (ALB) Routing Mechanics

An **Application Load Balancer (ALB)** functions at the Application Layer (Layer 7) of the Open Systems Interconnection (OSI) model. This positioning allows it to inspect incoming HTTP/HTTPS packets and make smart routing decisions based on URL paths, domain names, or header values.

When integrated with an Auto Scaling Group, the ALB plays an important role:

- **Dynamic Target Registration:** As the ASG scales out and launches new instances, it automatically registers their unique private IP addresses with the ALB's target group. Developers do not need to manually update configuration files or restart routing services.
- **Health Routing Isolation:** The ALB continuously sends small ping requests to each instance. If a server stops responding or encounters an internal software error, the load balancer flags it as unhealthy and stops sending traffic to it immediately, routing user requests to the remaining healthy servers instead.

4.3 CloudWatch Integration and Diagnostic Probing

Amazon CloudWatch serves as the primary monitoring and observability service for AWS infrastructure components. It works directly with the Auto Scaling Group by gathering performance data at regular intervals (such as standard 5-minute intervals or detailed 1-minute intervals).

CloudWatch uses custom alarms to automate scaling actions. When creating an alarm, administrators define a metric threshold and a duration rule (for example, *average CPU usage exceeding 70% for three consecutive evaluation periods*). This duration rule helps prevent the system from over-reacting to temporary performance spikes, ensuring the ASG only scales out when it detects sustained traffic changes.

4.4 Advanced Policies: Target Tracking and Step Scaling

To control how resources are added or removed, administrators can configure different types of scaling policies based on their workload patterns:

- **Target Tracking Scaling Policies:** The easiest policy type to manage. Administrators choose a specific target metric value, such as *maintaining an average CPU utilization of exactly 60%*. The scaling engine then

calculates how many instances to add or remove to keep the system stable around that target value.

- **Step Scaling Policies:** This approach responds to load changes using a series of stepped rules. For example, an administrator can configure a policy that adds **1 instance** if average CPU usage sits between 65% and 75%, but adds **3 instances** immediately if CPU usage spikes past 85%, providing a faster response to sudden traffic surges.

4.5 Scheduled and Predictive Scaling utilizing Machine Learning

While reactive scaling options work well for unexpected traffic changes, AWS also provides proactive scaling tools for predictable deployment patterns:

- **Scheduled Scaling:** This policy changes instance capacity based on known time schedules. It is ideal for predictable traffic patterns, such as scaling up an employee portal on Monday mornings or expanding an e-commerce platform's capacity ahead of a planned holiday sale.
- **Predictive Scaling:** This advanced approach uses machine learning algorithms to analyze historical traffic data and forecast upcoming resource needs. The system identifies recurring daily or weekly traffic cycles and schedules instance scaling actions proactively, ensuring new servers are fully booted and ready just before traffic increases.

4.6 IAM Frameworks and Network Isolation Security

Securing automated cloud infrastructure requires a multi-layered security approach using core AWS access controls:

- **AWS Identity and Access Management (IAM):** Used to enforce least-privilege access across the environment. Instead of using hardcoded API credentials, the Auto Scaling Group uses custom IAM Roles. These roles grant the scaling engine secure permission to manage EC2 instances and link with load balancers automatically.
- **Network Isolation and Security Groups:** To keep infrastructure secure, instances managed by an ASG should be deployed within private subnets to isolate them from direct public internet exposure. Security Groups act as virtual firewalls, configured to accept traffic exclusively from the public-facing Application Load Balancer while blocking all other external network requests.

4.7 Enterprise Production Scenarios and Industry Use Cases

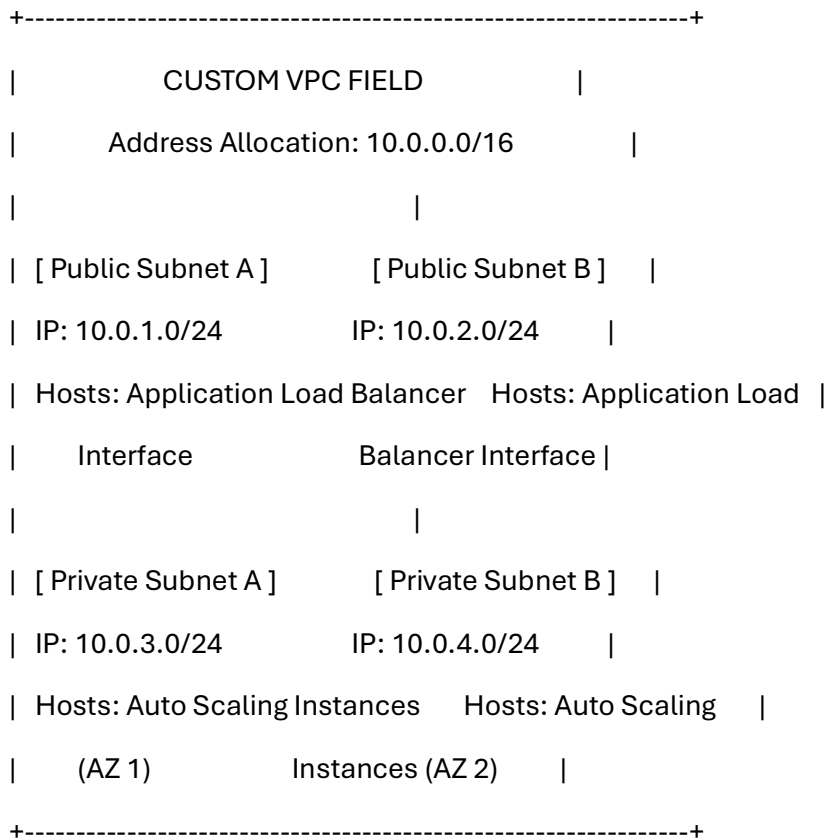
Automated cloud scaling models are widely used across modern industries to handle shifting infrastructure demands:

- **E-Commerce and Retail Platforms:** Online shopping websites experience massive traffic spikes during flash sales and product launches. Auto Scaling ensures these platforms expand capacity to process high order volumes smoothly, then scale back down to minimize infrastructure costs once the rush ends.
- **Media Streaming and Entertainment Services:** Video platforms experience higher user traffic during evening hours and weekends. Predictive scaling policies allow these platforms to scale up their streaming server pools ahead of time to ensure smooth video delivery.
- **Financial Processing and Batch Pipelines:** Banking systems process large volumes of transaction data overnight. Auto Scaling allows these institutions to spin up large compute pools to handle intense batch processing jobs rapidly, then terminate those servers as soon as processing finishes to keep operational costs low.

CHAPTER 5: PRACTICAL IMPLEMENTATION AND CONFIGURATION

5.1 Project Environment Requirements and Network Layout

This chapter outlines the step-by-step implementation of an automated, scalable web infrastructure environment completed during the Summer Internship Program. The base architecture was deployed within a secure, custom-designed cloud network built to the following specifications:



This layout ensures that the public internet can only access the front-facing load balancer, keeping the core application servers isolated in private subnets across multiple Availability Zones to maximize security and uptime.

5.2 Step-by-Step Production Security Group Creation

To secure network traffic, two separate EC2 Security Groups were configured to act as firewalls, controlling how data flows between the internet, the load balancer, and the application servers:

1. The Load Balancer Security Group (alb-security-group)

- **Inbound Rules:** Configured to accept incoming public web traffic on HTTP Port 80 and HTTPS Port 443 from any source IP address range (0.0.0.0/0).
- **Outbound Rules:** Allowed to send traffic on all ports out to any destination network.

2. The Application Server Security Group (app-node-security-group)

- **Inbound Rules:** Restricts incoming traffic by only accepting connections on HTTP Port 80 if they originate from the alb-security-group security signature. This rule prevents external users from bypassing the load balancer to access the backend servers directly.
- **Outbound Rules:** Configured to allow all outbound traffic so instances can download software patches and access external database connections.

5.3 Provisioning the Application Load Balancer Tier

With the core network and security rules in place, the next step involved setting up the Application Load Balancer to manage traffic routing:

1. Open the **AWS Management Console**, navigate to the **EC2 Dashboard**, select **Load Balancers**, and click **Create Load Balancer**.
2. Choose **Application Load Balancer** and set its scheme to **Internet-facing** so it can accept public user requests.
3. Under the Network Mapping options, select your custom VPC and bind the load balancer to **Public Subnet A** and **Public Subnet B** to establish an access path across multiple Availability Zones.
4. Set the Security Group selector to use your newly configured **alb-security-group**.
5. Under Listeners and Routing, create an HTTP Port 80 listener and configure it to forward incoming traffic toward a new target group named app-backend-pool. Set the target type to **Instances** and select HTTP health checks to monitor backend server stability.

5.4 Authoring the Launch Template with Scripted User Data

Next, a reusable configuration blueprint was designed to control how the Auto Scaling engine builds new server nodes:

1. In the EC2 Console, navigate to **Launch Templates** and click **Create template**.
2. Name the template **bca-asg-template-v1** and describe it as the "Production Web Server Blueprint."
3. Select the **Amazon Linux 2023 AMI** (64-bit architecture) to provide a stable operating system environment.
4. Set the compute profile to use the **t3.micro** instance type, keeping the deployment within the AWS Free Tier.
5. Link the template to your **app-node-security-group** to apply your custom firewall settings.
6. Open the **Advanced Details** panel, scroll to the **User Data** field, and input the following initialization script to handle software setup tasks automatically at boot time:

Bash

```
#!/bin/bash
```

```
sudo dnf update -y
```

```
sudo dnf install httpd -y
```

```
sudo systemctl start httpd
```

```
sudo systemctl enable httpd
```

```
TOKEN=$(curl -X PUT "http://169.254.169.254/latest/api/token" -H "X-aws-ec2-metadata-token-ttl-seconds: 21600")
```

```
INSTANCE_ID=$(curl -H "X-aws-ec2-metadata-token: $TOKEN" -s  
http://169.254.169.254/latest/meta-data/instance-id)
```

```
echo "<h1>BCA Final Project - Automated Server Node: $INSTANCE_ID</h1>" >  
/var/www/html/index.html
```

5.5 Deploying and Binding the Auto Scaling Group

The final step in the setup process was building the Auto Scaling Group itself and linking it with your network configuration and launch template:

1. In the EC2 Console, navigate to **Auto Scaling Groups** and click **Create Auto Scaling group**.

2. Name the group **bca-production-asg** and link it directly to your **bca-asg-template-v1** template.
3. On the Network Configuration page, select your custom VPC and choose **Private Subnet A** and **Private Subnet B** to ensure all automated instances are deployed securely away from direct public internet exposure.
4. Under Advanced Options, check **Attach to an existing load balancer** and select your **app-backend-pool** target group. Turn on **ELB Health Checks** and set the grace period to **300 seconds** to give instances enough time to boot before health tracking starts.
5. In the Group Size panel, set these scaling boundaries:
 - *Minimum capacity: 1*
 - *Desired capacity: 2*
 - *Maximum capacity: 4*
6. Choose **Target tracking scaling policy**, name it **cpu-target-60**, and set it to monitor **Average CPU Utilization** with a target value of **60%**. Click **Create** to initialize your automated server environment.

5.6 Workload Simulation and Stress-Testing Execution

To verify that the automated scaling environment works correctly under load, a stress test was executed to simulate a sudden surge in user traffic:

1. Log into one of your running backend EC2 instances securely using **AWS Systems Manager Session Manager** or an SSH connection.
2. Install the Linux stress-testing utility by running the following command in the terminal:

Bash

```
sudo dnf install stress -y
```

3. Run the utility to max out processing capacity across all system threads for 10 minutes:

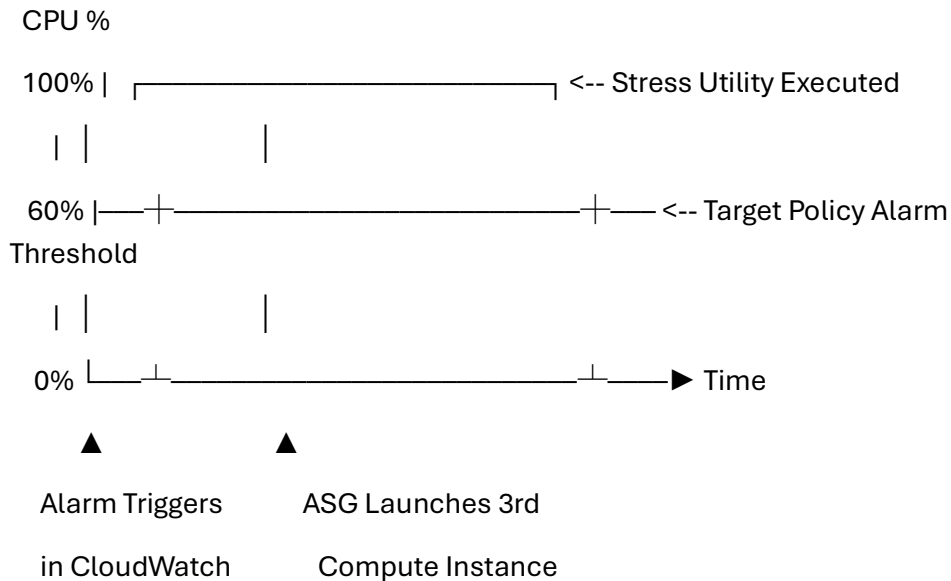
```
` `` bash
```

```
stress --cpu 4 --timeout 600
```

4. This command forces the server's CPU usage up to 100%, simulating an intense traffic spike. The monitoring tier will detect this usage surge and prepare to trigger a scaling event.

5.7 Analyzing Live Scale-Out Logs and CloudWatch Metrics

Once the stress test started, the system's response was tracked across different monitoring dashboards:



- **CloudWatch Tracking:** Within minutes, the CloudWatch dashboard showed average CPU utilization crossing the 60% alarm threshold, shifting the alarm status into an **In Alarm** state.
- **ASG Activity Logs:** Checking the *Activity History* tab in the Auto Scaling Group console confirmed that the alarm triggered a scale-out event. The log records showed the ASG automatically updating its desired capacity from 2 to 3 and launching a new instance using the `bca-asg-template-v1` template.
- **Load Balancer Validation:** As soon as the new instance passed its health checks, the ALB registered it automatically and began routing web traffic to it, proving the architecture successfully handles workload spikes without manual intervention.

CHAPTER 6: ADVANTAGES, LIMITATIONS, AND FUTURE SCOPE

6.1 Cost Optimization and Operational Advantages

Using an automated scaling architecture provides significant advantages for modern corporate enterprise deployments:

- **Lower Infrastructure Expenses:** Traditional hosting models require purchasing extra hardware to handle peak traffic loads, leading to high costs for servers that sit idle most of the time. AWS Auto Scaling keeps costs low by matching your running instance count with real-time user demand, ensuring you only pay for the exact compute capacity you use.
- **Improved Uptime and Reliability:** The ASG continuously monitors instance health. If a server encounters a software bug or a hardware failure, the system terminates the broken instance and launches a fresh one automatically, helping maintain application availability.
- **Reduced Manual Overhead:** System administrators no longer need to manually monitor server logs or provision extra resources during off-hours, allowing engineering teams to focus on core development tasks instead of infrastructure upkeep.

6.2 Architectural Vulnerabilities, Cooldown Caps, and Thrashing

Despite its obvious benefits, automated horizontal scaling requires careful design to avoid common operational issues:

- **The Boot Time Gap:** Launching a new virtual machine is not instantaneous; it can take several minutes for an instance to spin up, run its initialization scripts, and register with the load balancer. During this boot window, application performance may continue to suffer if traffic increases faster than the new servers can initialize.
- **Resource Thrashing:** If scaling policies are configured incorrectly—such as setting an alarm threshold too low or a cooldown window too short—the system can enter a loop called *thrashing*. This occurs when the ASG repeatedly launches and terminates instances in quick succession, leading to unstable application performance and unpredictable cloud costs.

6.3 Stateless vs. Stateful Application Disruption Barriers

A significant technical hurdle when adopting a horizontal scaling model is how an application handles user session data:

- **The Problem with Stateful Designs:** Legacy applications often store user login data, shopping cart contents, and session states locally within the server's local memory. In a horizontally scaling environment, if the load balancer routes a user's next request to a newly launched instance, the new server will not have access to that local memory, causing session drops or errors.
- **The Stateless Solution:** To scale out horizontally, software architectures must be redesigned to be stateless. This approach moves user session data out of individual server memory and into centralized, high-speed storage services like **Amazon ElastiCache (Redis)** or distributed databases like **Amazon DynamoDB**. This setup ensures all backend instances can access shared session data smoothly, allowing servers to be added or removed without disrupting the user experience.

6.4 Next-Generation Systems: Proactive AI-Driven Cloud Scaling

As cloud platforms continue to evolve, infrastructure management is moving away from reactive, threshold-based scaling controls toward proactive, artificial intelligence-driven systems.

Future innovations in AWS Auto Scaling focus on expanding **Predictive Scaling Engines**. By training machine learning models on years of historical application log data, these systems can forecast upcoming traffic surges hours before they occur. The predictive engine pre-provisions the required server instances ahead of time, entirely eliminating the performance drops associated with traditional reactive scaling boot delays.

Additionally, as modern development groups move toward containerized architectures managed via Amazon ECS/EKS and event-driven serverless computing via AWS Lambda, scaling mechanisms will operate at millisecond granularities. This evolution will ensure near-instant resource allocation while lowering operational costs to the absolute minimum.

CHAPTER 7: CONCLUSION AND REFERENCES

7.1 Comprehensive Technical Project Synthesis

This project report details the design, setup, and validation of automated, elastic cloud infrastructure built using **Amazon Web Services (AWS) Auto Scaling**. Transitioning from traditional, rigid hardware hypervisors to modern public cloud models solves long-standing enterprise engineering challenges around capacity planning, server availability, and high infrastructure costs.

The practical deployment completed during this Summer Internship Program demonstrates the power of closed-loop infrastructure automation. By combining an Application Load Balancer, EC2 instance pools, and Amazon CloudWatch metrics, we successfully engineered a system that adapts to shifting workloads automatically.

The simulated load tests proved that the system adds compute capacity smoothly during traffic spikes and removes extra servers once demand drops, protecting application performance while keeping costs under control. Ultimately, mastering automated cloud scaling techniques is an essential skill for modern computer science professionals, helping organizations build resilient platforms that operate efficiently in a digital economy.

7.2 Future Scope for Systems Research

While this project focuses on scaling standard Amazon EC2 virtual servers, there are several promising areas for future research and development:

- **Multi-Cloud Elasticity Orchestration:** Investigating tools that let architectures scale across multiple public cloud providers simultaneously (such as combining AWS and Google Cloud Platform), which can help protect large platforms against major single-provider network outages.
- **Intelligent Container Scaling:** Exploring how to optimize scaling rules in container environments by analyzing the relationship between Kubernetes pod scaling speeds and underlying cloud instance provisioning times.
- **Eco-Elasticity Frameworks:** Developing custom scaling algorithms that consider the carbon footprint and energy efficiency of cloud data centers, automatically routing intensive processing tasks to regions running on green, renewable energy during off-peak hours.

REFERENCES

1. **Comer, D. E.** (2021). *The Cloud Computing Book: The Technologies, Architecture, and Protocols Behind Cloud Systems*. CRC Press.
2. **Amazon Web Services Inc.** (2025). *Amazon EC2 Auto Scaling User Guide*. Official AWS Technical Documentation.
3. **Varia, J., & Mathew, S.** (2014). *Overview of Amazon Web Services: AWS Whitepaper*. AWS Documentation.
4. **Buyya, R., Vecchiola, C., & Selvi, S. T.** (2013). *Mastering Cloud Computing: Foundations and Applications Programming*. McGraw Hill Education.
5. **Velte, A. T., Velte, T. J., & Elsenpeter, R.** (2010). *Cloud Computing: A Practical Approach*. McGraw-Hill.
6. **Kaufman, L.** (2009). *Data Security in the World of Cloud Computing*. IEEE Security & Privacy, 7(4), 61-64.